

Desenvolvimento Iterativo, Colaboração com IA e Validação Automatizada

CampusFlow

Entrega 1 — Ambiente, Especificação Técnica e Test Harness

Integrante: Felipe Amorim Monteiro

RA: 22452139

Repositório público:

<https://github.com/MorimFr/entrega-inicial>

Data: 1 de setembro de 2026

RASCUNHO: adicionar as capturas do Docker, Issues, Pull Request e rulesets, e confirmar com o docente a revisão para equipe individual. Não submeter ao Moodle enquanto esta caixa existir.

1. Resumo do projeto

O CampusFlow é uma API de reserva de salas de estudo que evita conflitos de horário, ocupação acima da capacidade e excesso de reservas por usuário. A primeira iteração permite listar salas, criar/consultar/cancelar reservas e verificar disponibilidade.

2. Ambiente e arquitetura

A implementação usa Python 3.12 como versão-base, FastAPI/Pydantic nos contratos HTTP, serviço independente para regras de negócio e repositório em memória substituível. O ambiente é empacotado em Dockerfile e Compose; as versões são fixadas no `pyproject.toml`.

Camada	Responsabilidade
API	Rotas, validação dos contratos e tradução de erros para HTTP.
Serviço	Casos de uso, duração, capacidade, conflitos e limite diário.
Domínio	Entidades e estados sem dependência do framework.
Repositório	Porta de persistência e adaptador em memória na iteração inicial.

3. Especificação SDD

`docs/SPECIFICATION.md` contém requisitos RF-01 a RF-06, regras RN-01 a RN-09, requisitos não funcionais, contratos JSON/HTTP e matriz de rastreabilidade. A revisão dos testes refinou fuso horário, intervalos adjacentes, limite diário e cancelamento repetido; as mudanças constam em `docs/SPEC_CHANGELOG.md`.

4. Agente de IA

O Codex foi usado para apoiar decomposição, estrutura inicial e testes. O arquivo `AGENTS.md` mantém no repositório as regras SDD, limites arquiteturais e comandos de validação. Saídas de IA não são aceitas por afirmação: precisam passar pelo mesmo harness, Pull Request e revisão humana. Os prompts e o protocolo estão em `docs/AI_USAGE.md`.

5. Comandos do test harness

Ambiente padronizado

```
docker compose --profile test run --rm --build tests
```

Windows local

```
.\scripts\setup.ps1
.\scripts\test.ps1
```

O harness executa Ruff, testes unitários e testes HTTP. O Pytest falha abaixo de 90% de cobertura. O GitHub Actions executa os mesmos gates em Pull Requests e publica `test-output.log` e `coverage.xml`.

6. Log comprovado da execução local

Resultado: Ruff aprovado; 26 testes aprovados; cobertura total de 99,39%; exit code 0.

```
All checks passed!
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-8.4.1
collected 26 items

tests\api\test_reservations_api.py ..... [ 30%]
tests\unit\test_reservation_service.py ..... [100%]

Name                               Stmts   Miss  Cover
-----
campusflow\api.py                   46      0   100%
campusflow\domain.py                11      0   100%
campusflow\errors.py                18      0   100%
campusflow\repository.py            29      1    97%
campusflow\schemas.py              13      0   100%
campusflow\service.py               47      0   100%
-----
TOTAL                               164      1    99%
Required test coverage of 90% reached. Total coverage: 99.39%
===== 26 passed in 0.40s =====
```

Log integral versionado em `docs/evidence/test-run.md`. O Compose foi validado sintaticamente. A evidência de execução Docker deve ser adicionada quando o daemon estiver disponível.

GitHub Actions — Pull Request #10

Pipeline real aprovado em Ubuntu/Python 3.12.14: Ruff aprovado, 26 testes e cobertura de 99,48%.

Execução 33577382487 do workflow Test harness

```
All checks passed!  
platform linux -- Python 3.12.14, pytest-8.4.1  
collected 26 items  
Required test coverage of 90% reached. Total coverage: 99.48%  
===== 26 passed in 0.34s =====
```

7. Evidências externas pendentes

Docker: inserir print do comando Compose e dos 26 testes aprovados no contêiner.

GitHub Project: inserir Board com Issues, responsável e estados da Sprint 1.

Pull Requests: inserir PRs com comentários, CI e a forma de revisão autorizada pelo docente.

Proteção: inserir ruleset de main exigindo PR e status check.

8. Governança para equipe individual

O trabalho possui apenas um integrante. A exigência de aprovação “entre membros” não pode ser atendida literalmente sem fabricar identidade. Deve-se confirmar com o docente revisão por professor/monitor/colega autorizado ou uma dispensa formal, preservando Issues, PRs e CI. A autorização escolhida será registrada no PDF final.

9. Checklist final

- Nome e RA preenchidos.
- Link do repositório público preenchido.
- Especificação, ADRs, agente, ambiente e harness documentados.
- 26 testes e cobertura de 99,39% comprovados localmente.
- GitHub Actions aprovado em Python 3.12.14, com artefato de evidência.
- Pendente: capturas legíveis das Issues, do PR e dos rulesets.
- Pendente: evidência Docker e confirmação docente sobre revisão individual.